```
# © MIT Introduction to Deep Learning
# http://introtodeeplearning.com
```

Lab 05: Music Generation with RNNs

In this portion of the lab, we will explore building a Recurrent Neural Network (RNN) for music generation. We will train a model to learn the patterns in raw sheet music in ABC notation and then use this model to generate new music.

```
# Import Tensorflow 2.0
import tensorflow as tf

# Download and import the MIT Introduction to Deep Learning package
!pip install mitdeeplearning --quiet
import mitdeeplearning as mdl

# Import all remaining packages
import numpy as np
import os
import time
import functools
from IPython import display as ipythondisplay
from tqdm import tqdm
from scipy.io.wavfile import write
!apt-get install abcmidi timidity > /dev/null 2>&1


# Check that we are using a GPU, if not switch runtimes
  # using Runtime > Change Runtime Type > GPU
assert len(tf.config.list_physical_devices('GPU')) > 0
```

Dataset

We've gathered a dataset of thousands of Irish folk songs, represented in the ABC notation. Let's download the dataset and inspect it:

```
# Download the dataset
songs = mdl.lab1.load_training_data()

# Print one of the songs to inspect it in greater detail!
example_song = songs[800]
print("\nExample song: ")
print(example_song)
```

```
Found 817 songs in text

Example song:
X:15
T:Humours of Whiskey
Z: id:dc-slipjig-15
M:9/8
L:1/8
K:B Minor
gfe fBB fBB|gfe fBB fga|gfe fBB fBB|agf efd cBA:|!
d2e fdf ecA|d2e fed gfe|d2e fdf efg|agf efd cBA:|!
```

We can convert a song in ABC notation to an audio waveform and play it back. Be patient for this conversion to run, it can take some time.

```
# Step 1: Imports
from IPython.display import Audio, display
import os

def play_abc(abc_text, filename="song"):
    # Step 1: Save ABC text to file
    with open(f"{filename}.abc", "w") as f:
        f.write(abc_text)

    # Step 2: Convert ABC → MIDI
    os.system(f"abc2midi {filename}.abc -o {filename}.mid")

    # Step 3: Convert MIDI → WAV
    os.system(f"timidity {filename}.mid -Ow -o {filename}.wav")
```

```
        # Step 4: Play audio
        display(Audio(f"{filename}.wav"))

    play_abc(example_song)
```

0:00 / 0:37

We can easily convert a song in ABC notation to an audio waveform and play it back. Be patient for this conversion to run, it can take some time.

```
# Convert the ABC notation to audio file and listen to it
mdl.lab1.play_song(example_song)
```

0:00 / 0:37

One important thing to think about is that this notation of music does not simply contain information on the notes being played, but additionally there is meta information such as the song title, key, and tempo. How does the number of different characters that are present in the text file impact the complexity of the learning problem? This will become important soon, when we generate a numerical representation for the text data.

```
# Join our list of song strings into a single string containing all songs
songs_joined = "\n\n".join(songs)

# Find all unique characters in the joined string
vocab = sorted(set(songs_joined))
print("There are", len(vocab), "unique characters in the dataset")
```

```
There are 83 unique characters in the dataset
```

Process the dataset for the learning task

Let's take a step back and consider our prediction task. We're trying to train a RNN model to learn patterns in ABC music, and then use this model to generate (i.e., predict) a new piece of music based on this learned information.

Breaking this down, what we're really asking the model is: given a character, or a sequence of characters, what is the most probable next character? We'll train the model to perform this task.

To achieve this, we will input a sequence of characters to the model, and train the model to predict the output, that is, the following character at each time step. RNNs maintain an internal state that depends on previously seen elements, so information about all characters seen up until a given moment will be taken into account in generating the prediction.

Vectorize the text

Before we begin training our RNN model, we'll need to create a numerical representation of our text-based dataset. To do this, we'll generate two lookup tables: one that maps characters to numbers, and a second that maps numbers back to characters. Recall that we just identified the unique characters present in the text.

```
### Define numerical representation of text ###

# Create a mapping from character to unique index.
# For example, to get the index of the character "d",
#   we can evaluate `char2idx["d"]`.
char2idx = {u:i for i, u in enumerate(vocab)}

# Create a mapping from indices to characters. This is
#   the inverse of char2idx and allows us to convert back
#   from unique index to the character in our vocabulary.
idx2char = np.array(vocab)
```

This gives us an integer representation for each character. Observe that the unique characters (i.e., our vocabulary) in the text are mapped as indices from 0 to `len(unique)`. Let's take a peek at this numerical representation of our dataset:

```
print('{')
for char,_ in zip(char2idx, range(82)):
```

```
    print('  {:4s}: {:3d},'.format(repr(char), char2idx[char]))
print('  ...\n}')
```

```
{
  '\n':   0,
  ' ' :   1,
  '!' :   2,
  '"' :   3,
  '#' :   4,
  "'" :   5,
  '(' :   6,
  ')' :   7,
  ',' :   8,
  '-' :   9,
  '.' :  10,
  '/' :  11,
  '0' :  12,
  '1' :  13,
  '2' :  14,
  '3' :  15,
  '4' :  16,
  '5' :  17,
  '6' :  18,
  '7' :  19,
  '8' :  20,
  '9' :  21,
  ':' :  22,
  '<' :  23,
  '=' :  24,
  '>' :  25,
  'A' :  26,
  'B' :  27,
  'C' :  28,
  'D' :  29,
  'E' :  30,
  'F' :  31,
  'G' :  32,
  'H' :  33,
  'I' :  34,
  'J' :  35,
  'K' :  36,
  'L' :  37,
  'M' :  38,
  'N' :  39,
  'O' :  40,
  'P' :  41,
  'Q' :  42,
  'R' :  43,
  'S' :  44,
  'T' :  45,
  'U' :  46,
  'V' :  47,
  'W' :  48,
  'X' :  49,
  'Y' :  50,
  'Z' :  51,
  '[' :  52,
  ']' :  53,
  '^' :  54,
  '_' :  55,
  'a' :  56,
```

```
### Vectorize the songs string ###

'''TODO: Write a function to convert the all songs string to a vectorized
    (i.e., numeric) representation. Use the appropriate mapping
    above to convert from vocab characters to the corresponding indices.

  NOTE: the output of the `vectorize_string` function
  should be a np.array with `N` elements, where `N` is
  the number of characters in the input string
'''
def vectorize_string(text):
  return np.array([char2idx[char] for char in text])

vectorized_songs = vectorize_string(songs_joined)
```

We can also look at how the first part of the text is mapped to an integer representation:

```
print ('{} ---- characters mapped to int ----> {}'.format(repr(songs_joined[:10]), vectorized_songs[:10]))
# check that vectorized_songs is a numpy array
assert isinstance(vectorized_songs, np.ndarray), "returned result should be a numpy array"
```

```
'X:1\nT:Alex' ---- characters mapped to int ----> [49 22 13  0 45 22 26 67 60 79]
```

Create training examples and targets

Our next step is to actually divide the text into example sequences that we'll use during training. Each input sequence that we feed into our RNN will contain `seq_length` characters from the text. We'll also need to define a target sequence for each input sequence, which will be used in training the RNN to predict the next character. For each input, the corresponding target will contain the same length of text, except shifted one character to the right.

To do this, we'll break the text into chunks of `seq_length+1`. Suppose `seq_length` is 4 and our text is "Hello". Then, our input sequence is "Hell" and the target sequence is "ello".

The batch method will then let us convert this stream of character indices to sequences of the desired size.

```
### Batch definition to create training examples ###

def get_batch(vectorized_songs, seq_length, batch_size):
  # the length of the vectorized songs string
  n = vectorized_songs.shape[0] - 1
  # randomly choose the starting indices for the examples in the training batch
  idx = np.random.choice(n-seq_length, batch_size)

  '''TODO: construct a list of input sequences for the training batch'''
  input_batch = [vectorized_songs[i : i+seq_length] for i in idx]
  '''TODO: construct a list of output sequences for the training batch'''
  output_batch = [vectorized_songs[i+1 : i+seq_length+1] for i in idx]


  # x_batch, y_batch provide the true inputs and targets for network training
  x_batch = np.reshape(input_batch, [batch_size, seq_length])
  y_batch = np.reshape(output_batch, [batch_size, seq_length])
  return x_batch, y_batch


# Perform some simple tests to make sure your batch function is working properly!
test_args = (vectorized_songs, 10, 2)
if not mdl.lab1.test_batch_func_types(get_batch, test_args) or \
   not mdl.lab1.test_batch_func_shapes(get_batch, test_args) or \
   not mdl.lab1.test_batch_func_next_step(get_batch, test_args):
   print("======\n[FAIL] could not pass tests")
else:
   print("======\n[PASS] passed all tests!")
```

```
[PASS] test_batch_func_types
[PASS] test_batch_func_shapes
[PASS] test_batch_func_next_step
======
[PASS] passed all tests!
```

For each of these vectors, each index is processed at a single time step. So, for the input at time step 0, the model receives the index for the first character in the sequence, and tries to predict the index of the next character. At the next timestep, it does the same thing, but the RNN considers the information from the previous step, i.e., its updated state, in addition to the current input.

We can make this concrete by taking a look at how this works over the first several characters in our text:

```
x_batch, y_batch = get_batch(vectorized_songs, seq_length=5, batch_size=3)

for i, (input_idx, target_idx) in enumerate(zip(np.squeeze(x_batch), np.squeeze(y_batch))):
    print("Step {:3d}".format(i))
    print("  input: {} ({:s})".format(input_idx, repr(idx2char[input_idx])))
    print("  expected output: {} ({:s})".format(target_idx, repr(idx2char[target_idx])))
```

```
Step   0
  input: [60 56  1 56 62] (array(['e', 'a', ' ', 'a', 'g'], dtype='<U1'))
  expected output: [56  1 56 62 60] (array(['a', ' ', 'a', 'g', 'e'], dtype='<U1'))
Step   1
  input: [ 1 56 14 82 56] (array([' ', 'a', '2', '|', 'a'], dtype='<U1'))
  expected output: [56 14 82 56 16] (array(['a', '2', '|', 'a', '4'], dtype='<U1'))
Step   2
  input: [56 65 70 73  0] (array(['a', 'j', 'o', 'r', '\n'], dtype='<U1'))
  expected output: [65 70 73  0 59] (array(['j', 'o', 'r', '\n', 'd'], dtype='<U1'))
```

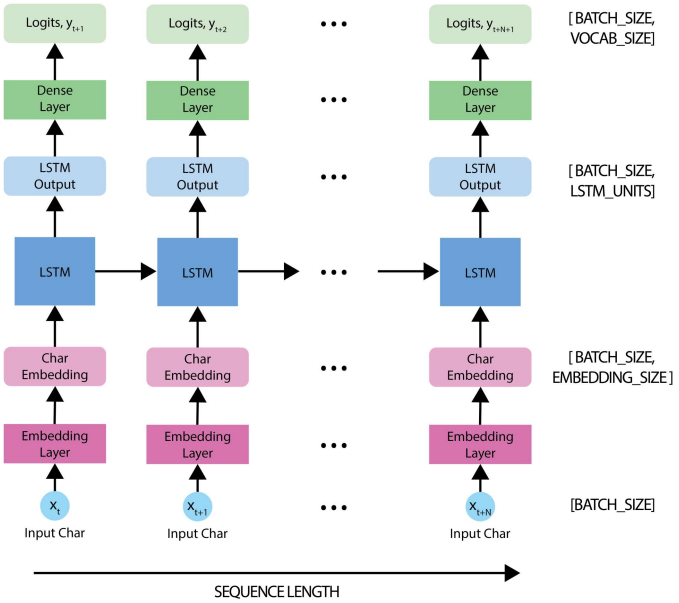
The Recurrent Neural Network (RNN) model

Now we're ready to define and train a RNN model on our ABC music dataset, and then use that trained model to generate a new song. We'll train our RNN using batches of song snippets from our dataset, which we generated in the previous section.

The model is based off the LSTM architecture, where we use a state vector to maintain information about the temporal relationships between consecutive characters. The final output of the LSTM is then fed into a fully connected `Dense` layer where we'll output a softmax over each character in the vocabulary, and then sample from this distribution to predict the next character.

As we introduced in the first portion of this lab, we'll be using the Keras API, specifically, `tf.keras.Sequential`, to define the model. Three layers are used to define the model:

- `tf.keras.layers.Embedding`: This is the input layer, consisting of a trainable lookup table that maps the numbers of each character to a vector with `embedding_dim` dimensions.
- `tf.keras.layers.LSTM`: Our LSTM network, with size `units=rnn_units`.
- `tf.keras.layers.Dense`: The output layer, with `vocab_size` outputs.



Define the RNN model

Now, we will define a function that we will use to actually build the model.

```
def LSTM(rnn_units):
  return tf.keras.layers.LSTM(
    rnn_units,
    return_sequences=True,
    recurrent_initializer='glorot_uniform',
    recurrent_activation='sigmoid',
    stateful=True,
  )
```

The time has come! Fill in the `TODOs` to define the RNN model within the `build_model` function, and then call the function you just defined to instantiate the model!

```
### Defining the RNN Model ###

'''TODO: Add LSTM and Dense layers to define the RNN model using the Sequential API.'''
def build_model(vocab_size, embedding_dim, rnn_units, batch_size):
  model = tf.keras.Sequential([
    # Layer 1: Embedding layer to transform indices into dense vectors
    #   of a fixed embedding size
    tf.keras.layers.Embedding(vocab_size, embedding_dim),

    # Layer 2: LSTM with `rnn_units` number of units.
    # TODO: Call the LSTM function defined above to add this layer.
    LSTM(rnn_units),


    # Layer 3: Dense (fully-connected) layer that transforms the LSTM output
    #   into the vocabulary size.
    # TODO: Add the Dense layer.
    tf.keras.layers.Dense(vocab_size)

  ])

  return model

# Build a simple model with default hyperparameters. You will get the
#   chance to change these later.
```

```
model = build_model(len(vocab), embedding_dim=256, rnn_units=1024, batch_size=32)
model.build(tf.TensorShape([32, 100]))  # [batch_size, sequence_length]
```

Test out the RNN model

It's always a good idea to run a few simple checks on our model to see that it behaves as expected.

First, we can use the `Model.summary` function to print out a summary of our model's internal workings. Here we can check the layers in the model, the shape of the output of each of the layers, the batch size, etc.

↳ 3 cells hidden

Predictions from the untrained model

Let's take a look at what our untrained model is predicting.

To get actual predictions from the model, we sample from the output distribution, which is defined by a `softmax` over our character vocabulary. This will give us actual character indices. This means we are using a [categorical distribution](https://en.wikipedia.org/wiki/Categorical_distribution) to sample over the example prediction. This gives a prediction of the next character (specifically its index) at each timestep.

Note here that we sample from this probability distribution, as opposed to simply taking the `argmax`, which can cause the model to get stuck in a loop.

Let's try this sampling out for the first example in the batch.

↳ 4 cells hidden

Training the model: loss and training operations

Now it's time to train the model!

At this point, we can think of our next character prediction problem as a standard classification problem. Given the previous state of the RNN, as well as the input at a given time step, we want to predict the class of the next character -- that is, to actually predict the next character.

To train our model on this classification task, we can use a form of the `crossentropy` loss (negative log likelihood loss). Specifically, we will use the `sparse_categorical_crossentropy` loss, as it utilizes integer targets for categorical classification tasks. We will want to compute the loss using the true targets -- the `labels` -- and the predicted targets -- the `logits`.

Let's first compute the loss using our example predictions from the untrained model:

```
### Defining the loss function ###

'''TODO: define the loss function to compute and return the loss between
    the true labels and predictions (logits). Set the argument from_logits=True.'''
def compute_loss(labels, logits):
  loss = tf.keras.losses.sparse_categorical_crossentropy(labels, logits, from_logits=True)
  return loss

'''TODO: compute the loss using the true next characters from the example batch
    and the predictions from the untrained model several cells above'''
# example_batch_loss = compute_loss(tf.squeeze(y), pred) # Commented out to resolve NameError

# print("Prediction shape: ", pred.shape, " # (batch_size, sequence_length, vocab_size)")
# print("scalar_loss:      ", example_batch_loss.numpy().mean()) # Commented out to resolve NameError
```

```
'TODO: compute the loss using the true next characters from the example batch\n    and the predictions from the untrained m
odel several cells above'
```

Let's start by defining some hyperparameters for training the model. To start, we have provided some reasonable values for some of the parameters. It is up to you to use what we've learned in class to help optimize the parameter selection here!

```
### Hyperparameter setting and optimization ###

vocab_size = len(vocab)

# Model parameters:
params = dict(
  num_training_iterations = 3000,  # Increase this to train longer
  batch_size = 8,  # Experiment between 1 and 64
  seq_length = 100,  # Experiment between 50 and 500
  learning_rate = 5e-3,  # Experiment between 1e-5 and 1e-1
  embedding_dim = 256,
```

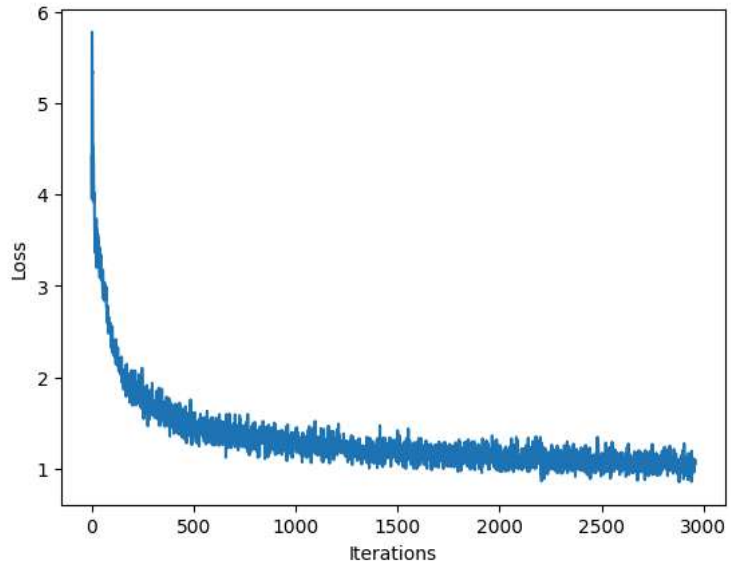
```
    rnn_units = 1024,  # Experiment between 1 and 2048
  )

  # Checkpoint location:
  checkpoint_dir = './training_checkpoints'
  checkpoint_prefix = os.path.join(checkpoint_dir, "my_ckpt.weights.h5")
  os.makedirs(checkpoint_dir, exist_ok=True)
```

Now, we are ready to define our training operation -- the optimizer and duration of training -- and use this function to train the model. You will experiment with the choice of optimizer and the duration for which you train your models, and see how these changes affect the network's output. Some optimizers you may like to try are `Adam` and `Adagrad`.

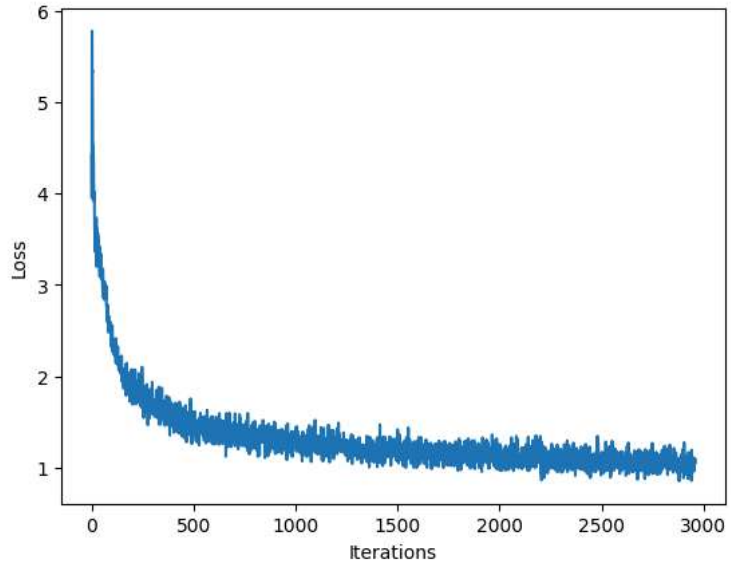
First, we will instantiate a new model and an optimizer. Then, we will use the `tf.GradientTape` method to perform the backpropagation operations.

We will also generate a print-out of the model's progress through training, which will help us easily visualize whether or not we are minimizing the loss.

```
### Define optimizer and training operation ###

'''TODO: instantiate a new model for training using the `build_model`
  function and the hyperparameters created above.'''
model = build_model(vocab_size, params["embedding_dim"], params["rnn_units"], params["batch_size"])

'''TODO: instantiate an optimizer with its learning rate.
  Checkout the tensorflow website for a list of supported optimizers.
  https://www.tensorflow.org/api_docs/python/tf/keras/optimizers/
  Try using the Adam optimizer to start.'''
optimizer = tf.keras.optimizers.Adam(learning_rate=params["learning_rate"])



@tf.function
def train_step(x, y):
  # Use tf.GradientTape()
  with tf.GradientTape() as tape:

    '''TODO: feed the current input into the model and generate predictions'''
    y_hat = model(x)

    '''TODO: compute the loss!'''
    loss = compute_loss(y, y_hat)

  # Now, compute the gradients
  '''TODO: complete the function call for gradient computation.
      Remember that we want the gradient of the loss with respect all
      of the model parameters.
      HINT: use `model.trainable_variables` to get a list of all model
      parameters.'''
  grads = tape.gradient(loss, model.trainable_variables)

  # Apply the gradients to the optimizer so it can update the model accordingly
  optimizer.apply_gradients(zip(grads, model.trainable_variables))
  return loss

##################
# Begin training!#
##################

history = []
plotter = mdl.util.PeriodicPlotter(sec=2, xlabel='Iterations', ylabel='Loss')

if hasattr(tqdm, '_instances'): tqdm._instances.clear() # clear if it exists
for iter in tqdm(range(params["num_training_iterations"])):

  # Grab a batch and propagate it through the network
  x_batch, y_batch = get_batch(vectorized_songs, params["seq_length"], params["batch_size"])
  loss = train_step(x_batch, y_batch)

  # Update the progress bar and also visualize within notebook
  history.append(loss.numpy().mean())
  plotter.plot(history)

  # Update the model with the changed weights!
  if iter % 100 == 0:
    model.save_weights(checkpoint_prefix)

# Save the trained model and the weights
model.save_weights(checkpoint_prefix)
```




```
100%|██████████| 3000/3000 [01:54<00:00, 26.24it/s]
```



Generate music using the RNN model

Now, we can use our trained RNN model to generate some music! When generating music, we'll have to feed the model some sort of seed to get it started (because it can't predict anything without something to start with!).

Once we have a generated seed, we can then iteratively predict each successive character (remember, we are using the ABC representation for our music) using our trained RNN. More specifically, recall that our RNN outputs a `softmax` over possible successive characters. For inference, we iteratively sample from these distributions, and then use our samples to encode a generated song in the ABC format.

Then, all we have to do is write it to a file and listen!

Restore the latest checkpoint

To keep this inference step simple, we will use a batch size of 1. Because of how the RNN state is passed from timestep to timestep, the model will only be able to accept a fixed batch size once it is built.

To run the model with a different `batch_size`, we'll need to rebuild the model and restore the weights from the latest checkpoint, i.e., the weights after the last checkpoint during training:

```
'''TODO: Rebuild the model using a batch_size=1'''

model = build_model(vocab_size, params["embedding_dim"], params["rnn_units"], batch_size=1)

# Restore the model weights for the last checkpoint after training
model.build(tf.TensorShape([1, None]))
model.load_weights(checkpoint_prefix)
```

```
model.summary()
```

Model: "sequential_6"

Layer (type)	Output Shape	Param #
embedding_7 (Embedding)	(1, None, 256)	21,248
lstm_6 (LSTM)	(1, None, 1024)	5,246,976
dense_6 (Dense)	(1, None, 83)	85,075

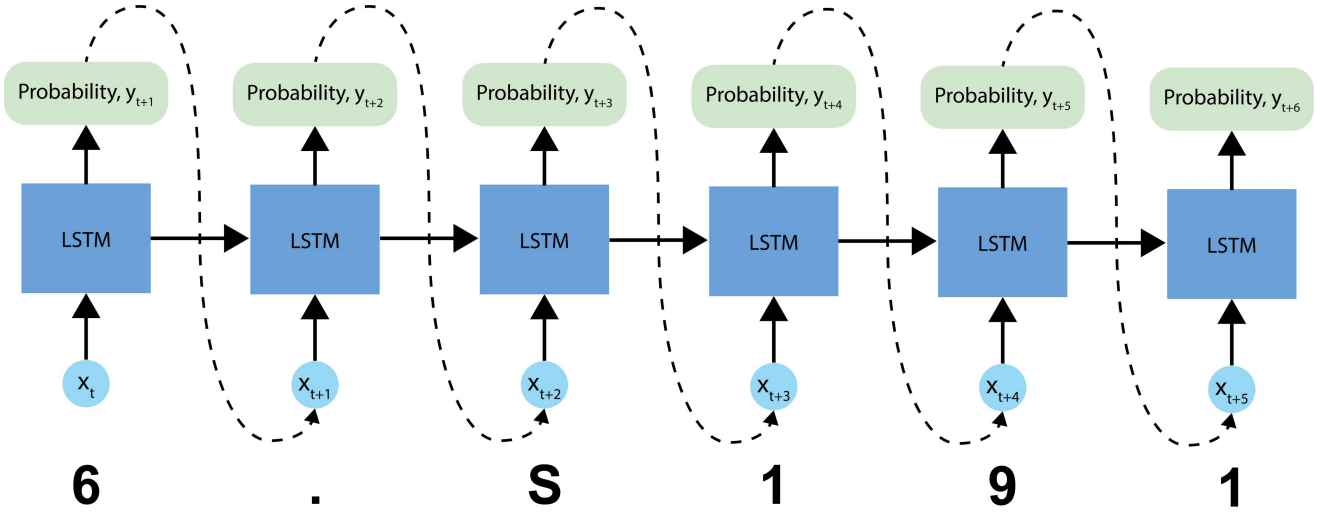
Total params: 5,353,299 (20.42 MB)
Trainable params: 5,353,299 (20.42 MB)
Non-trainable params: 0 (0.00 B)

Notice that we have fed in a fixed `batch_size` of 1 for inference.

The prediction procedure

Now, we're ready to write the code to generate text in the ABC music format:

- Initialize a "seed" start string and the RNN state, and set the number of characters we want to generate.
- Use the start string and the RNN state to obtain the probability distribution over the next predicted character.
- Sample from multinomial distribution to calculate the index of the predicted character. This predicted character is then used as the next input to the model.
- At each time step, the updated RNN state is fed back into the model, so that it now has more context in making the next prediction. After predicting the next character, the updated RNN states are again fed back into the model, which is how it learns sequence dependencies in the data, as it gets more information from the previous predictions.



Complete and experiment with this code block (as well as some of the aspects of network definition and training!), and see how the model performs. How do songs generated after training with a small number of epochs compare to those generated after a longer duration of training?

```
### Prediction of a generated song ###
temperature = 0.8
def generate_text(model, start_string, generation_length=1000):
  # Evaluation step (generating ABC text using the learned RNN model)

  '''TODO: convert the start string to numbers (vectorize)'''
  input_eval = [char2idx[s] for s in start_string]

  input_eval = tf.expand_dims(input_eval, 0)

  # Empty string to store our results
  text_generated = []

  # Here batch size == 1
  for layer in model.layers:
    if hasattr(layer, "reset_states"):
        layer.reset_states()
  tqdm._instances.clear()

  for i in tqdm(range(generation_length)):
      '''TODO: evaluate the inputs and generate the next character predictions'''
      predictions = model(input_eval)
```

```
        # Remove the batch dimension
        predictions = tf.squeeze(predictions, 0)
        predictions = predictions / temperature
        '''TODO: use a multinomial distribution to sample'''
        predicted_id = tf.random.categorical(predictions, num_samples=1)[-1,0].numpy()

        # Pass the prediction along with the previous hidden state
        #   as the next inputs to the model
        input_eval = tf.expand_dims([predicted_id], 0)

        '''TODO: add the predicted character to the generated text!'''
        # Hint: consider what format the prediction is in vs. the output
        text_generated.append(idx2char[predicted_id])


    return (start_string + ''.join(text_generated))
```

```
'''TODO: Use the model and the function defined above to generate ABC format text of length 1000!
    As you may notice, ABC files start with "X" - this may be a good start string.'''
generated_text = generate_text(model, start_string="X", generation_length=1000)
```

```
100%|██████████| 1000/1000 [00:13<00:00, 75.08it/s]
```

Play back the generated music!

We can now call a function to convert the ABC format text to an audio file, and then play that back to check out our generated music! Try training longer if the resulting song is not long enough, or re-generating the song!

```
### Play back generated songs ###

generated_songs = mdl.lab1.extract_song_snippet(generated_text)
saved_files = []
for i, song in enumerate(generated_songs):
  # Synthesize the waveform from a song
  waveform = mdl.lab1.play_song(song)

  # If its a valid song (correct syntax), lets play it!
  if waveform:
    print("Generated song", i)
    ipythondisplay.display(waveform)

    numeric_data = np.frombuffer(waveform.data, dtype=np.int16)
    wav_file_path = f"output_{i}.wav"
    write(wav_file_path, 88200, numeric_data)

    saved_files.append(wav_file_path)

# Show download links (for Jupyter/Colab)
for file in saved_files:
    print("Saved:", file)
```

```
Found 2 songs in text
Generated song 1

      0:00 / 4:00

Saved: output_1.wav
```

Experiment!

Congrats on making your first sequence model in TensorFlow! It's a pretty big accomplishment, and hopefully you have some sweet tunes to show for it.

Consider how you may improve your model and what seems to be most important in terms of performance. Here are some ideas to get you started:

- How does the number of training epochs affect the performance?
- What if you alter or augment the dataset?
- Does the choice of start string significantly affect the result?

Try to optimize your model and generate your best song!

Experimenting with Training Iterations and Temperature

To experiment, you can modify the following:

1. **Number of Training Iterations**: Adjust the `num_training_iterations` value in the `params` dictionary within the 'Hyperparameter setting and optimization' cell (`JQWUUhKotkAY`). Increase it for more training or decrease it for quicker (but potentially less refined) results.
2. **Prediction Temperature**: Change the `temperature` variable in the `generate_text` function definition (`WvuwZBX5Ogfd`).
    - A higher `temperature` (e.g., 1.0 or more) will make the generated music more random and creative.
    - A lower `temperature` (e.g., 0.1-0.5) will make the generated music more predictable and closer to the training data.

After making changes to either of these, **you must re-run the relevant cells**:

- If you change `num_training_iterations`, re-run the training cell (`F31vzJ_u66cb`).
- After training, always re-run the model rebuilding cell for inference (`LycQ-ot_jjyu`) and the generation cell (`ktovv0RFhrkn`) to see the new music.